

20/08/2025

Clase 02: SCM – Software Configuration Management

Es una disciplina que integra la Ingeniería de Software, nos sirve para mantener la integridad del software como producto ya que este es maleable, por lo que será necesario para afrontar los cambios que se presentarán.



- **Proceso:** Es una definición conceptual o teórica de lo que hay que hacer. Describe qué cosas se deben hacer, en qué orden, qué artefactos producir, y cómo se deben llevar a cabo las tareas.
- **Proyecto:** Es una **unidad de gestión** o de organización del trabajo. Es el medio que permite organizar a las personas y administrar los recursos para obtener un resultado. El proceso se instancia en un proyecto, esto significa que las tareas teóricas definidas en el proceso se ejecutan concretamente en el proyecto.
 - Tiene un resultado único, cada producto es distinto.
 - Tiene un esfuerzo temporal definido, o sea, una fecha de inicio y fecha de fin.
 - Es de elaboración gradual, el objetivo se descompone y se alcanza de a poco a través de tareas interrelacionadas.
- **Producto:** Es el resultado o la salida de un proyecto y del proceso de desarrollo de software.

¿Qué es el **Software**?

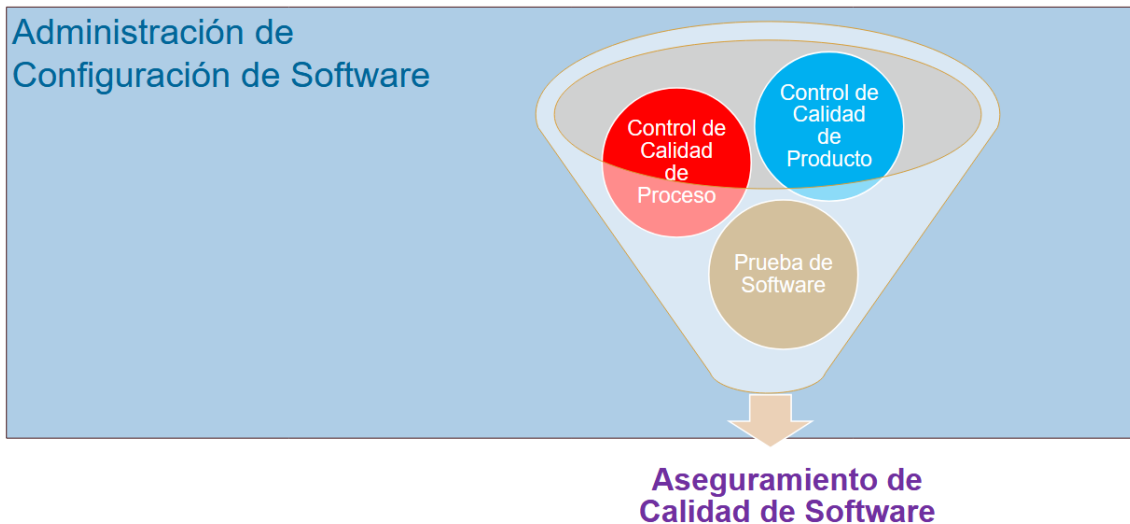
Consiste en un conjunto de programas, procedimientos, reglas, documentación y datos. Es información estructurada con propiedades lógicas y funcionales, creada y mantenida en varias formas y representaciones, diseñada para ser procesada por una computadora.

¿Qué produce los **Cambios en el Software**?

- Cambios del negocio y nuevos requerimientos.
- Soporte de cambios de productos asociados.

- Reorganización de las prioridades de la empresa por crecimiento.
- Cambios en el presupuesto.
- Defectos encontrados a corregir.
- Oportunidades de mejora.

Disciplinas de soporte del Software



No se debe confundir **Aseguramiento de Calidad de Software** con **Control de Calidad**, el aseguramiento busca la calidad antes de que el producto esté terminado. Estos permiten, mediante la definición de sus procesos, asegurar que los procesos que se ejecutan generen productos que sean de calidad.

¿Qué significa que un **producto sea íntegro**?

- Satisface las necesidades del usuario.
- Puede ser fácil y completamente rastreado durante su ciclo de vida.
- Satisface criterios de performance.
- Cumple con sus expectativas de costo.

Y, ¿Qué **problemas** se presentan cuando **maneja componentes**?

- Pérdida de un componente
- Pérdida de cambios (el componente que tengo no es el último)
- Sincronía fuente-objeto-ejecutable
- Regresión de fallas
- Doble mantenimiento
- Superposición de cambios
- Cambios no validados

Conceptos clave para SCM

- Ítem de Configuración
- Repositorio
- Línea Base
- Ramas (Branch)
- Configuración del Software

Ítem de Configuración

Son todos y cada uno de los artefactos que forman parte del producto o del proyecto, que pueden sufrir cambios o necesitan ser compartidos entre los miembros del equipo y sobre los cuales necesitamos conocer su estado y evolución.

¿Qué puede ser un Ítem de Configuración?

Es cualquier cosa que se pueda meter en un filesystem. Hay un listado, este es:

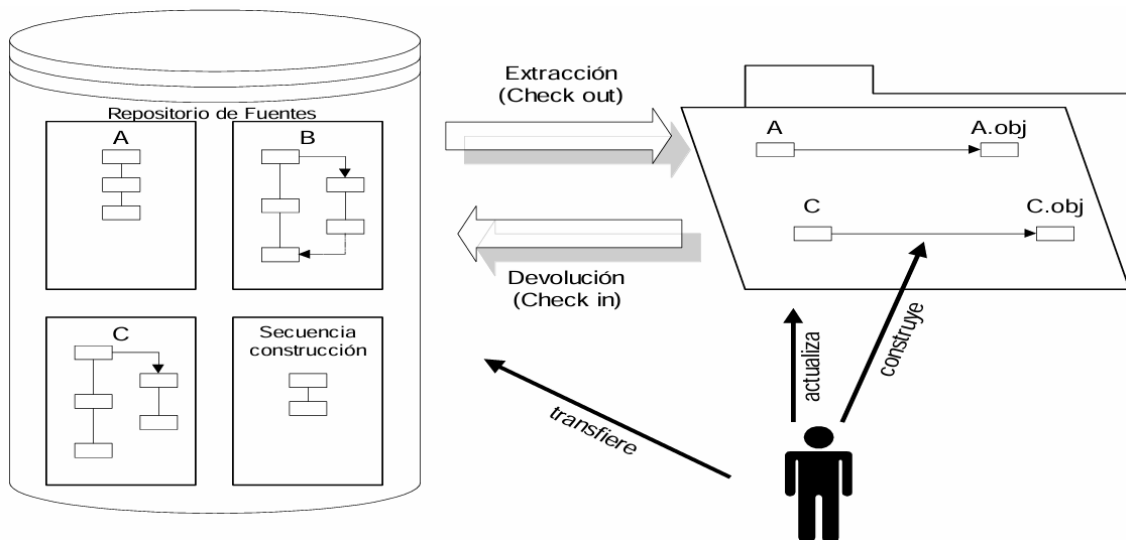
- | | |
|-----------------------------|-------------------------------|
| ➤ Plan de CM | ➤ Planes de Iteración |
| ➤ Propuestas de Cambio | ➤ Estándares de codificación |
| ➤ Visión | ➤ Casos de prueba |
| ➤ Riesgos | ➤ Código fuente |
| ➤ Plan de desarrollo | ➤ Gráficos, iconos, ... |
| ➤ Prototipo de Interfaz | ➤ Instructivo de ensamble |
| ➤ Guía de Estilo de IHM | ➤ Programa de instalación |
| ➤ Manual de Usuario | ➤ Documento de despliegue |
| ➤ Requerimientos | ➤ Lista de Control de entrega |
| ➤ Plan de Calidad | ➤ Formulario de aceptación |
| ➤ Arquitectura del Software | ➤ Registro del proyecto |
| ➤ Plan de Integración | |

Versión y Variante

- Una versión se define, desde el punto de vista de la evolución, como la forma particular de un artefacto en un instante o contexto dado.
- Una variante es una versión de un ítem de configuración (o de la configuración) que evoluciona por separado.

Repositorio

Un repositorio es un contenedor de información que almacena los ítems de configuración (ICs). Puede ser un directorio, una base de datos (relacional o de objetos), o una combinación de ambas.



- **Check-out** (Extracción): Permite obtener la última versión (o versiones anteriores) de los ítems de configuración y traerlos a un directorio local para su modificación.
- **Check-in** (Devolución): Sirve para introducir las versiones modificadas de los ítems de configuración de vuelta al repositorio.

Tipos de repositorios:

- **Centralizados:** Un único servidor contiene todos los archivos con sus versiones. Los administradores tienen mayor control, pero una falla en el servidor puede ser crítica
 - Las acciones de check-in y check-out se limitan a la última versión o a lo que se va a cambiar.
- **Descentralizados:** Cada cliente tiene una copia idéntica del repositorio completo. Esto ofrece mayor resiliencia (si un servidor falla, se puede "copiar y pegar" desde un cliente) y posibilita otros flujos de trabajo.
 - Permiten traer una versión completa del repositorio, lo cual es más costoso en espacio, pero más cómodo.

Línea Base

Una línea base es una configuración del software que ha sido formalmente revisada y acordada, sirviendo como un punto seguro y una base para desarrollos posteriores. Se considera una versión íntegra de un conjunto de ítems de configuración en un momento determinado.

Ramas

Creación de ramas

- Existe una rama principal (trunk, master).
- Sirven para bifurcar el desarrollo.
- Pueden tener razones de creación con semántica.

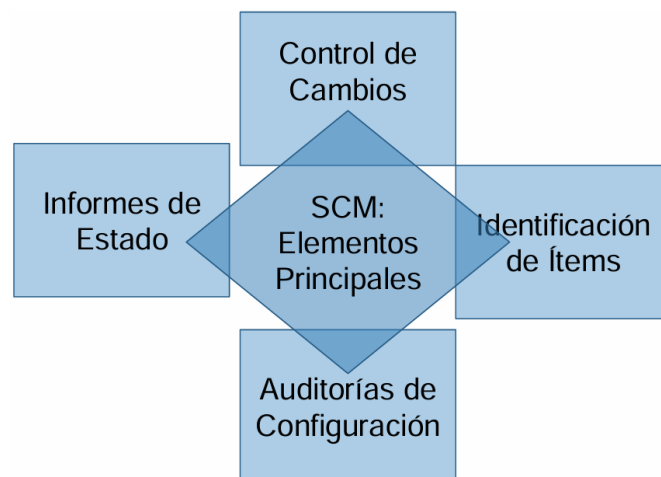
- Permiten la experimentación.
- Pueden ser descartadas o integradas.

Integración de ramas

- La operación se llama merge.
- Lleva los cambios a la rama principal.
- Pueden surgir conflictos (resolvemos con diff).
- Todas las ramas deberían eventualmente integrarse a la principal o ser descartadas.

Configuración del Software

Actividades Fundamentales de la Gestión de Configuración del Software



Identificación de Ítems

Se define la estructura del repositorio, se identifican ítems unívocamente y se asignan a un lugar específico en la estructura del repositorio. Se definen también reglas de nombrado, convenciones, esquemas genéricos, etc.

Tipos de Ítems de Configuración



Los Ítems de Configuración de Iteración son de los que duran menos su ciclo de vida, ya que una vez que se termina la iteración estos dejan de ser tan relevantes.

Los relacionados al producto son los que más van a trascender a lo largo de los proyectos (un proyecto por cada iteración).

Control de Cambios

Un proceso de control de cambios es un proceso donde, a partir de una línea base, se pide hacer un control de cambios, para ello:

1. **Propuesta de Cambio** (o Petición de Cambios): Se genera una propuesta que detalla el cambio solicitado.
2. **Análisis de Impacto**: Se evalúan los aspectos técnicos, los efectos colaterales, los costos del proyecto y el impacto general que el cambio podría tener.
3. **Revisión de Partes**: La propuesta y el análisis de impacto son revisados por las partes interesadas.
4. **Decisión del Comité de Control de Cambios (CCC)**: Este comité se encarga de analizar el cambio y su impacto para aceptar o rechazar la modificación.
5. **Notificación a Partes / Informar al Cliente**: Una vez tomada la decisión, se notifica a todas las partes involucradas, incluyendo al cliente.
6. **Generación de Orden de Cambios de Ingeniería**: En caso de aprobación, se genera una orden formal para implementar el cambio.
7. **Ubicación en la cola de cambios**: El cambio aprobado se coloca en una lista para su implementación.
8. **Verificación y Validación**: Una vez implementado el cambio, se verifica que se haya realizado correctamente y que el software resultante cumpla con los requerimientos esperados. Solo después de este paso, se podría establecer una nueva línea base.

Comité de Control de Cambios

- **Comité de Control de Cambios**: Gestiona cambios menores e incluye roles clave del equipo de desarrollo, como el analista que define la solución, el desarrollador que la implementará, el líder técnico y el líder de testing.
- **Comité de Control de Cambios Ampliado**: Interviene en cambios mayores que tienen un impacto significativo (económico, de producto a largo plazo, etc.). Este comité incluye figuras como el gerente de calidad o el responsable del producto, además de los líderes de proyecto.

Auditorías de Configuración

- **Auditoría Física de Configuración** (PCA - Physical Configuration Audit): Asegura que lo que está indicado para cada Ítem de Configuración (IC) en la línea base o en la actualización se ha alcanzado realmente. Se centra en la verificación de que el producto se construye de la manera correcta.
- **Auditoría Funcional de Configuración** (FCA - Functional Configuration Audit): Es una evaluación independiente de los productos de software, controlando que la funcionalidad y el rendimiento reales de cada ítem de configuración sean consistentes con la especificación de requerimientos. Abarca tanto la verificación

(el producto se construye correctamente) como la validación (el producto correcto resuelve los problemas del cliente).

Estas auditorías tienen 2 características:

- **Objetividad:** Para que el proceso sea objetivo, la persona que ejecuta la auditoría debe ser externa al proyecto sobre el que se realiza la auditoría. Puede ser alguien de la organización, pero que no forme parte del equipo del proyecto. A menudo, el gerente de configuración lidera estas auditorías o se contrata a una auditora externa.
- **Resultados:** La salida de un proceso de auditoría es un informe de auditoría que contiene un listado de hallazgos (fallas, cosas a corregir), oportunidades de mejora y observaciones. También puede identificar cosas que se hicieron correctamente.

Informes de Estado

Su función principal es mantener los registros de la evolución del sistema. Estos informes manejan una gran cantidad de información y, por lo tanto, suelen implementarse dentro de procesos automáticos. Incluyen reportes de rastreabilidad de todos los cambios realizados a las líneas base durante el ciclo de vida del producto.

Algunas preguntas que un Informe de Estado puede responder:

- ¿Cuáles el estado del ítem?
- ¿Un requerimiento de cambio ha sido aprobado o rechazado por el CCB?
- ¿Qué versión de ítem implementa un requerimiento de cambio aprobado (saber cuál es el componente que contiene la mejora)?
- ¿Cuál es la diferencia entre una versión y otra dada?

Preguntas sobre la aplicación en Agile, según Google NotebookLM:

¿Qué pasa con el Comité de Control de Cambios?

En los ambientes Ágiles, la existencia y el funcionamiento del Comité de Control de Cambios es "medio discutido". La filosofía Ágil promueve responder a los cambios en lugar de tratar de evitarlos, y valora la colaboración con el cliente más que las negociaciones contractuales formales. El SCM en Ágil debe servir al equipo de desarrollo y coordinar el desarrollo en lugar de controlar a los desarrolladores. El proceso tradicional de pedir aprobación formal a un comité para cada cambio se considera contrario a la agilidad. En su lugar, la interacción y el feedback continuo con el cliente son fundamentales; si al cliente le gusta un cambio, se aprueba mediante esa colaboración y no a través de un proceso burocrático, priorizando mostrar software funcionando.

¿Qué ítems de configuración podemos tener?

En el contexto de SCM en Ágil, si bien la definición de ICs se mantiene, hay un fuerte enfoque en la "Documentación Lean y Trazabilidad" y en "Eliminar el desperdicio - no

agregar nada más que valor". Esto implica que la elección de qué ítems de configuración se mantendrán y con qué nivel de detalle se hará, buscará la eficiencia y la utilidad directa para el equipo y el cliente, priorizando el software funcionando sobre una documentación excesiva. Sin embargo, elementos esenciales como el código fuente, los requerimientos (a menudo expresados como User Stories), y las pruebas seguirán siendo ítems de configuración críticos.

¿Qué pasa con las auditorías?

Aunque los materiales no afirman que las auditorías desaparecen por completo, el enfoque Ágil prioriza el "Feedback continuo y visible sobre calidad, estabilidad e integridad" y la automatización de tantos procesos como sea posible. La naturaleza iterativa y de inspección-adaptación de los procesos empíricos (base de Ágil) sugiere que los controles de calidad y la verificación de la integridad se integran de manera continua en el desarrollo, en lugar de depender de auditorías formales y periódicas realizadas por terceros. La idea de "no hacer cosas de más" y "no documentar de más" implica una simplificación de procesos que no añaden valor directo o que pueden ser abordados de manera más ágil.

¿Qué pasa con los reportes de estado?

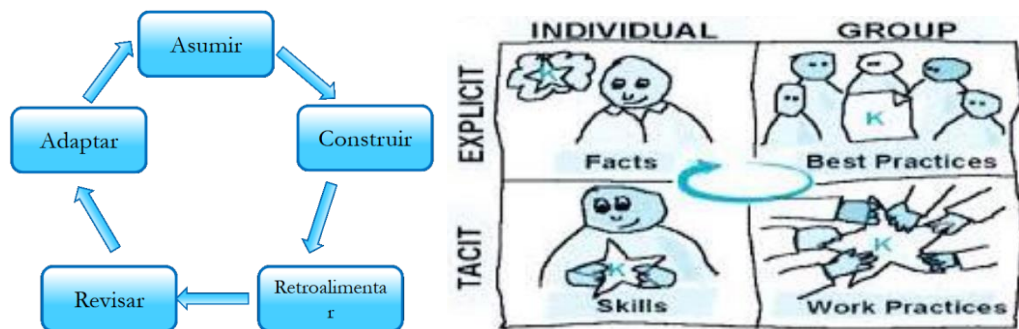
El agilismo busca eliminar el desperdicio y promover la "Documentación Lean y Trazabilidad". En lugar de informes formales y extensos, se valora el feedback continuo y visible sobre calidad, estabilidad e integridad. La automatización juega un papel clave, permitiendo que las herramientas de gestión de configuración proporcionen información en tiempo real sobre el estado del software. La idea es generar rápidamente versiones nuevas del software que puedan ser mostradas al cliente y que "realmente anden", en lugar de depender de informes detallados. Esto significa que el enfoque cambia de la generación manual de informes a la obtención de información de estado de manera más dinámica y automatizada, integrada en el flujo de trabajo continuo del equipo.

Clase 03: Filosofía Ágil y Manifiesto Ágil

Procesos empíricos

Los procesos empíricos son un tipo de enfoque para el desarrollo de software que se basa en la experiencia, el aprendizaje continuo y la adaptación constante, a diferencia de los procesos definidos que buscan la previsibilidad a través de pasos detallados y formales.

Patrón de conocimiento en los procesos empíricos



1. **Asumir:** Se parte de una hipótesis o de lo que se cree que es necesario hacer.
2. **Construir:** Se desarrolla el producto o se realiza el trabajo basado en esa asunción.
3. **Retroalimentar:** Se recibe el feedback o la realimentación sobre el trabajo que se ha construido. El feedback continuo y visible sobre la calidad, estabilidad e integridad del software es un aspecto clave de la inspección.
4. **Revisar:** Se procede a la revisión del trabajo realizado. Esto implica inspeccionar qué se hizo bien y qué se hizo mal, aprendiendo de la experiencia generada. La transparencia es crucial en esta etapa para que la inspección sea efectiva, implicando ser sincero y compartir información, sin ocultar problemas y pidiendo ayuda cuando sea necesario.
5. **Adaptar:** Como consecuencia de la inspección, se realizan los ajustes necesarios al proceso o al producto.

El manifiesto ágil

Los 4 valores ágiles

- Individuos e interacciones sobre procesos y herramientas: Esto significa que, aunque los procesos y las herramientas son valiosos, se prioriza a las personas y las relaciones interpersonales entre ellas.
- Software funcionando sobre documentación exhaustiva: Se valora más lo que entrega valor al cliente, es decir, el software que funciona, por encima de grandes cantidades de documentación detallada.
- Colaboración con el cliente sobre negociación contractual: Se prefiere una relación de colaboración integrada con el cliente, trabajando juntos, en lugar de aferrarse a contratos firmados y negociaciones formales.

- Respuesta al cambio sobre seguir un plan rígido: Se valora la capacidad de entender que habrá cambios y de incorporarlos, en lugar de aferrarse estrictamente a un plan predefinido.

Los 12 principios ágiles



¿Qué es **ágil**?

Es una ideología, **no** una metodología **ni** un proceso.

¿Qué significa **ser ágil**?

- Busca un **equilibrio entre la ausencia total de procesos y el exceso de formalidad y burocracia**.
- Los métodos ágiles son **adaptables** en lugar de predictivos, lo que significa que están diseñados para reaccionar y ajustarse a los cambios.
- Son **orientados a las personas** en lugar de orientados al proceso, priorizando las interacciones humanas.

¿Qué **métodos ágiles** hay disponibles?

- Scrum: Es un framework ágil muy conocido. Se menciona que tiene tres roles, tres categorías de responsabilidades, cuatro ceremonias o reuniones y tres artefactos. Es tan exigente que, si no se hace al pie de la letra de lo que dice, no se garantiza los resultados.
- XP (eXtreme Programming): Fue el primer framework que surgió en la industria. Se menciona que tiene 14 principios, y si no se cumplen, no se considera "programación extrema" y no se garantizan los resultados.

- Crystal: Este es un conjunto de prácticas que se especializan según el tipo de software que se va a construir. Por ejemplo, indica diferentes prácticas si el software es crítico (pone en riesgo vidas), solo afecta el confort, o involucra grandes cantidades de dinero.
- FDD (Feature Driven Development).
- ATDD (Acceptance Test Driven Development): Este enfoque lleva el Test Driven Development (desarrollo guiado por pruebas) hacia las pruebas de negocio, de modo que las prácticas de desarrollo están dirigidas a demostrar que los casos de negocio funcionan en el software.
- Shap: Este es un framework más reciente, mencionado como una propuesta que surge en "rebeldía" de Scrum.

29/08/2025

Clase 03 P1: Estimaciones ágiles

¿Qué es la estimación?

Es intentar predecir el futuro en términos de esfuerzo, tiempo y costo para construir un producto de software.

Se suele confundir con la planificación, pero mientras que la estimación es una técnica o conjunto de técnicas utilizadas, la planificación implica un compromiso de negocio y no necesariamente debe estar directamente relacionada con la estimación.

Las estimaciones nos brindan beneficios:

- Sirven como una protección para el equipo al establecer una base para el compromiso.
- Permiten determinar la factibilidad de un trabajo planificado.
- Lo más beneficioso en las estimaciones es el “proceso de hacerlas”.

Estimaciones Tradicionales vs Estimaciones Ágiles

¿Cómo se toman las estimaciones tradicionales?

- **Tamaño del producto:** Se estima en términos de casos de uso o líneas de código.
- **Esfuerzo:** Se mide en horas lineales (también conocidas como horas-hombre).
- **Tiempo calendario:** Traducción de las horas lineales a días/meses, que depende de:
 - Personas involucradas
 - Horario de trabajo de cada persona
 - Habilidades de las personas
 - Herramientas disponibles
 - Superposición de tareas
- **Costo:** El mayor costo en el desarrollo de software es la mano de obra.

¿Y las estimaciones ágiles?

- Se utilizan **medidas relativas** en lugar de absolutas.
- Surgen de la **comparación** y el acuerdo dentro del equipo
- Son **entendibles solo en el contexto del equipo** específico que las realiza

Ventajas de la **estimación relativa**

- Las personas no saben estimar en términos absolutos
- Somos buenos comparando cosas
- Comparar es generalmente más rápido.

- Se obtiene una mejor dinámica grupal y pensamiento de equipo más que individual
- Se emplea mejor el tiempo de análisis de las stories

La unidad de medida para las estimaciones relativas son los “Story Points” (SP), esta unidad de medida tiene la característica de que no se basa en el tiempo, sino que depende de las habilidades, conocimiento, experiencia y familiaridad del equipo con el dominio de aplicación/negocio. Esta unidad representa la cantidad de trabajo para producir una story basándose en lo decidido por el equipo, que se considera:

- Complejidad: Cuán compleja es la story
- Esfuerzo: Cuánto trabajo se necesita para completarla.
- Incertidumbre: Cuánta duda hay sobre lo que se conoce o no de la user story.

Escalas de los Story Points

Para medir qué tan grande es la story, hacemos uso de escalas, las más comunes de usar son:

- Tamaño por números: Del 1 al 10.
- Talles de remeras: S, M, L, XL, XXL
- Serie 2^{na}: 1, 2, 4, 8, 16, 32, 64, etc.
- Sucesión de Fibonacci: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, etc.

La técnica de estimación que vamos a usar: Poker Planning

Mediante el uso de cartas, cada participante tendrá un set de cartas (en este caso) con la sucesión de Fibonacci y la lista de stories a estimar, lo que se va a hacer a continuación será:

En caso de que sea la primera vez que se hace la Poker Planning, el primer paso será determinar una User Story canónica (base para la comparación), asignándole un valor de 1 o 2 SP, una vez hecho eso, se hará lo siguiente:

1. Leer y discutir la User Story a estimar, haciendo preguntas al Product Owner.
2. Cada estimador selecciona una carta (con un valor en SP) y la pone boca abajo.
3. Cuando todos las ponen, se exponen al mismo tiempo.
4. Si hay consenso, ese es el estimado. Si no, los estimadores con los valores más altos y bajos explican su razonamiento.
5. Se repiten las rondas de discusión y votación hasta que el número converge.

El valor que se decida indicará el tamaño, como referencias será:

0: No hay idea clara, se necesita más información (similar a un signo de pregunta).

1/2, 1: Funcionalidad muy pequeña, ideal para historias canónicas.

2, 3, 5: Valores ideales para historias de tamaño pequeño a mediano.

8: Funcionalidad grande, se debe considerar si puede dividirse.

13, 20, 40, 100: Valores muy grandes que indican que la user story no es estimable, no cumplirá con el criterio "Small" del modelo INVEST y debe ser particionada.

Un derivado de estas estimaciones es el concepto de Velocidad (Velocity), esta es una métrica de progreso del equipo, calculada sumando los Story Points de las User Stories completadas en una iteración, sirve para corregir errores de estimación a lo largo del tiempo y tiende a ser inestable en los primeros sprints, ya que el equipo está aprendiendo a trabajar en conjunto. Gracias a la velocidad podemos definir la capacidad del equipo para el siguiente sprint.

Las User Stories se desglosan en tareas con estimaciones en horas dentro del Sprint Backlog.

Clase 03 P2: Gestión de Productos

La Gestión de Producto de Software se centra en el ciclo de vida de un producto desde su concepción hasta su declive, enfocándose en la entrega de **valor al cliente** y la **validación de hipótesis de mercado**.

Propósito y Enfoque Central

- **Motivaciones para la Creación de Productos:** Los productos de software se crean para diversas razones, como satisfacer a los clientes, atraer a muchos usuarios, generar ingresos significativos o cumplir una gran visión para cambiar el mundo.
- **Enfoque en el Valor:** Es fundamental entregar **valor** al cliente, no solo líneas de código. El software es un medio para entregar valor de negocio.
 - Se estima que el 80% de las funcionalidades de un producto suelen no aportar valor, por lo que la gestión de producto busca concentrarse en el 20% que sí lo hace.
 - La productividad no se mide por cuánto se construye, sino por descubrir lo *correcto* a construir.
 - El éxito radica en entregar un producto o característica que el cliente **usará** y que esté alineado continuamente con sus necesidades reales.
- **Ciclo de Vida del Producto:** Un producto de software atraviesa fases de Introducción, Crecimiento, Madurez y Declive. A diferencia de un proyecto, un producto nunca está realmente "terminado".

Evolución del Valor en los Productos de Software Los productos de software pueden evolucionar a través de distintos niveles de valor:

- **Funcional:** El producto resuelve un problema específico.
- **Confiable:** Se puede usar de manera consistente.

- **Usable:** Es fácil de usar y aprender.
- **Conveniente:** Es accesible y se integra fácilmente en la vida del usuario.
- **Placentero:** Evoca emociones positivas y deleite en el usuario.
- **Significativo:** Tiene un impacto profundo y transformador.

Cruzar de los niveles funcionales a los placenteros o significativos es un desafío importante para las organizaciones.

Conceptos Clave en la Gestión de Producto

La gestión de producto utiliza varios conceptos para desarrollar y validar ideas de manera eficiente:

1. MVP (Minimum Viable Product - Producto Mínimo Viable):

- **Definición:** Es la versión de un nuevo producto creada con el **menor esfuerzo posible** para **validar una hipótesis de mercado** y recopilar la máxima cantidad de aprendizaje validado sobre los clientes.
- **Objetivo:** Confirmar si una idea es atractiva o útil para el público antes de invertir grandes recursos.
- **Características:** Debe tener suficiente valor para que las personas estén dispuestas a usarlo o comprarlo, demostrar un beneficio futuro para retener a los primeros usuarios y proporcionar un ciclo de retroalimentación.
- **Formato:** No necesariamente es un producto funcional desplegable en producción. Puede ser una demo, un video o un prototipo que simula la funcionalidad, siempre que los clientes potenciales lo perciban como un producto que funciona.
- **Proceso:** Implica un ciclo de retroalimentación de "Construir-Experimentar-Aprender" (Build-Experiment-Learn Feedback Loop).
- **Importancia:** Permite "pivotar" (cambiar la dirección del producto) si la hipótesis inicial no se valida completamente.
- **Errores comunes:** Confundir MVP con características o productos comercializables mínimos, enfatizar solo lo "mínimo" sin ser "viable" o de calidad suficiente, o ignorar el feedback después del lanzamiento.

2. MLP (Minimum Lovable Product - Producto Mínimo Adorable):

- **Definición:** Es la versión más simple de un producto que no solo resuelve una necesidad, sino que también **genera una conexión emocional o deleite en los usuarios**, buscando su entusiasmo por comprarlo.
- **Objetivo:** Ir más allá de la funcionalidad para apelar a la emocionalidad, generar lealtad y emociones positivas.

- **Elementos clave:** Además de funcionalidad y usabilidad, incluye el **deleite** a través de interacciones intuitivas y fomento de la lealtad.
- **Ejemplos:** Spotify, Instagram, WhatsApp son productos que lograron una alta adhesión emocional.

3. MVF (Minimal Viable Feature - Característica Mínima Viable):

- **Definición:** Una mini versión del MVP, es una característica a pequeña escala construida rápidamente con recursos mínimos para probar su utilidad y adopción.
- **Uso:** Se utiliza cuando hay alta incertidumbre sobre una característica específica pero alto potencial.

4. MMF (Minimal Marketable Feature - Característica Mínima Comercializable):

- **Definición:** La pieza más pequeña de funcionalidad que puede ser liberada, y que tiene valor tanto para la organización como para los usuarios.
- **Objetivo:** Aportar valor y generar crecimiento cuando hay alta certeza sobre su valor.

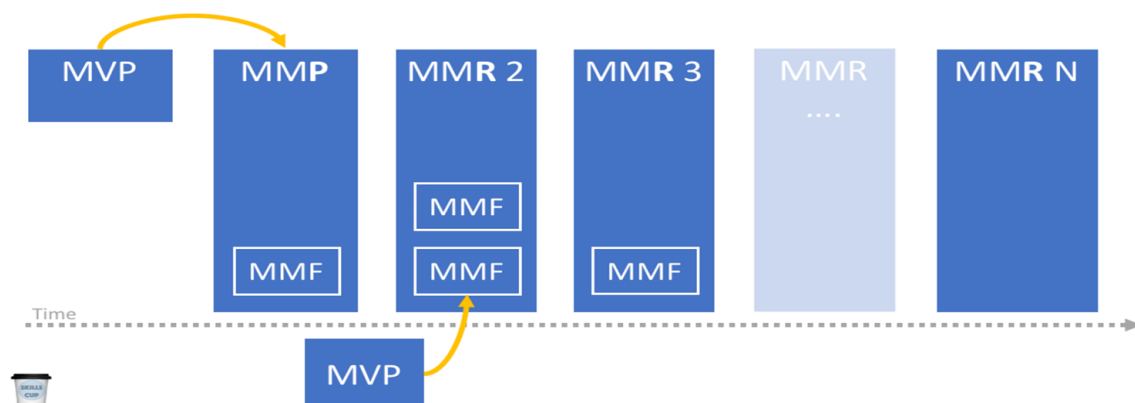
5. MMR (Minimal Marketable Release - Lanzamiento Mínimo Comercializable):

- **Definición:** Un lanzamiento de un producto con el conjunto más pequeño posible de características que ofrece un valor nuevo a los usuarios y satisface sus necesidades actuales.

6. MMP (Minimal Marketable Product - Producto Mínimo Comercializable):

- **Definición:** El primer release de un MMR, dirigido a early adopters, enfocado en características clave que satisfarán a este grupo.

Relación entre MVP, MMF, MMP y MMR



- MVP: para validar hipótesis con el mínimo esfuerzo.
- MMF: funcionalidad mínima liberable con valor claro.
- MMP: primer release con lo mínimo para early adopters.
- MMR: lanzamiento con un set mínimo que entrega valor real.

Otros Conceptos Importantes

- **Audacia de Cero:** más fácil conseguir inversión con “cero” (clientes/ingresos) que con pocos, pero retrasar un MVP aumenta riesgo y desperdicio.
- **Supuestos de Saltos de Fe:**
 - Hipótesis críticas que sostienen todo el plan.
 - Se validan con **hipótesis de valor** (retención) y **hipótesis de crecimiento** (referencias, NPS).

05/09/2025

Clase 04: Componentes de un Proyecto de SW y Estimaciones tradicionales

El Proceso de Software y la Gestión de Proyectos

Un proceso de software es un conjunto estructurado de actividades que se realizan para desarrollar o mantener un sistema. Para que sea efectivo, se necesita la “mesa de tres patas”:

- Procedimientos y métodos
- Herramientas y equipos
- Personas con habilidades, entrenamiento y motivación

Existen dos enfoques principales:

Procesos definidos: buscan repetibilidad y predictibilidad. Más que “iguales entradas → iguales salidas”, se describen como procesos con **entradas identificadas y salidas esperadas**. Valorán la experiencia como extrapolable a otros contextos y los cambios requieren **evaluación y autorización formal**. Su administración se basa en la planificación y control formales.

Procesos empíricos: reconocen la variabilidad y la necesidad de inspección frecuente y adaptación. Funcionan mejor en contextos creativos y cambiantes. El patrón empírico es: asumir → construir → retroalimentar → revisar → adaptar. Permiten que el equipo defina cambios en el proceso y buscan hacer explícito el conocimiento implícito para que no se pierda. Son compatibles únicamente con **ciclos de vida iterativos**.

¿Qué es un Proyecto?

Un proyecto de software es un esfuerzo temporal con un resultado único, definido por:

- Orientación a objetivos: los guía y dan sentido al trabajo.
- Duración limitada: empieza y termina al alcanzar los objetivos.
- Tareas interrelacionadas: requieren coordinación de esfuerzos y recursos.
- Unicidad: cada proyecto es distinto, incluso si parece similar a otro. Esta es su característica más relevante.

Ciclos de Vida de un Proyecto de Software

El ciclo de vida organiza las fases del proceso y define el orden en que se realizan.

Tipos básicos:

- Secuencial: fases una detrás de otra (por ej. cascada).
- Iterativo: se repite en ciclos más pequeños, incorporando retroalimentación.
- Recursivo: implica profundizar o reexplorar fases anteriores varias veces.

Las actividades esenciales (requerimientos, análisis, diseño, implementación, pruebas, operación y mantenimiento) siempre están presentes; lo que varía es cómo se organizan y coordinan.

Administración de Proyectos

La administración de proyectos consiste en aplicar conocimientos, habilidades, herramientas y técnicas para cumplir objetivos en tiempo y presupuesto, satisfaciendo las especificaciones.

Sus pilares son:

- Planificación: definir objetivos, alcance, recursos, riesgos y métricas.
- Seguimiento y control: comparar lo planificado con lo real, detectar desvíos y corregirlos.

Es importante remarcar que **no puede haber seguimiento sin un plan**, porque el seguimiento es un control de lo planificado.

La Triple Restricción

También llamada “triángulo de hierro”, combina:

- Alcance: qué se debe hacer.
- Tiempo: cuánto tardará.
- Costos: cuánto costará.

El balance entre estas tres dimensiones determina la calidad del proyecto. El líder de proyecto es quien debe mantener este equilibrio.

La diferencia entre enfoques es clave:

- En la **gestión tradicional**, se fija el alcance (ERS) y de ahí se derivan tiempo y costo. Los cambios son posibles, pero implican un proceso formal de análisis y autorización.
- En la **gestión ágil**, se fijan tiempo (iteraciones) y recursos/costos (equipo dedicado), y el alcance se negocia continuamente con el Product Owner mediante la priorización del Product Backlog.

El Rol del Líder y del Equipo

- Líder de proyecto: equilibra la triple restricción, coordina a stakeholders, cliente, subcontratistas y equipo, y es responsable del éxito global. En la gestión tradicional asigna trabajo con un sistema **push**, mientras que en enfoques ágiles los equipos autogestionados usan un sistema **pull**, eligiendo sus propias tareas.
- Equipo de proyecto: grupo pequeño con habilidades diversas, comprometido con objetivos comunes. La efectividad depende de la sinergia, la responsabilidad compartida y la buena comunicación.

El Plan de Proyecto

Es la hoja de ruta del proyecto, debe responder:

- ¿Qué vamos a hacer?
- ¿Cuándo lo haremos?
- ¿Cómo lo haremos?
- ¿Quién lo hará?

Debe ser un documento **vivo**, que guíe diariamente, y no un requisito formal olvidado.

El Alcance

- Alcance del producto: características y funciones que debe tener el software.
- Alcance del proyecto: todo el trabajo necesario para entregar el producto, incluyendo actividades como **definir requerimientos, diseñar la arquitectura, programar y testear**.

Su cumplimiento se mide contra la especificación de requerimientos (producto) y contra el plan de proyecto (trabajo).

Estimaciones

Las estimaciones son intentos de predecir el futuro en términos de tamaño, esfuerzo, calendario, costo y recursos críticos.

Lo más valioso de estimar no es el número final, sino el proceso de hacerlo, ya que obliga a entender, discutir y analizar el proyecto.

En la gestión tradicional, se realizan así:

- Tamaño: contando requerimientos como casos de uso ajustados por complejidad. No se recomienda usar líneas de código en proyectos nuevos.
- Esfuerzo: medido en horas-persona lineales, resultando en una “bolsa de horas”.
- Calendario: conversión de la bolsa de horas en tiempo real, considerando personas, horas productivas, solapamiento de tareas y feriados.
- Costo: horas-persona por costo del ingeniero, más un porcentaje para gastos adicionales (infraestructura, licencias, etc.).

En lo tradicional, estima el líder o un experto, mientras que en enfoques ágiles las estimaciones son **relativas** y las realiza el propio equipo que ejecutará las tareas.

Gestión de Riesgos

Un riesgo es un problema esperando suceder.

La gestión incluye:

1. Identificación de riesgos.
2. Análisis de probabilidad e impacto.

3. Definir mitigación y planes de contingencia.
4. Monitorear y actualizar riesgos durante todas las fases.

Los riesgos deben registrarse en una lista de riesgos para ser reutilizados como base de conocimiento en futuros proyectos. Ejemplos típicos son la **volatilidad de requerimientos, pérdida de personal clave, problemas tecnológicos y errores de estimación.**

Métricas

Sirven para evaluar objetivamente el estado del proyecto.

Tipos:

- De proceso: consolidan información a nivel organizacional.
- De proyecto: seguimiento específico del proyecto.
- De producto: calidad, tamaño, defectos, cobertura de pruebas.

Distintos niveles requieren métricas diferentes:

- Desarrollador: esfuerzo, duración de tareas, defectos en pruebas unitarias.
- Equipo: tamaño del producto, distribución del esfuerzo, volatilidad de requerimientos.
- Organización: cumplimiento de cronograma, precisión de estimaciones, desempeño en presupuesto.

Se recomienda **mantenerlas simples** y nunca utilizarlas para castigar o premiar a las personas, ya que evalúan al producto, proyecto o proceso, no a los individuos.

Monitoreo y Control

Un proyecto se atrasa “un día a la vez”.

Factores clave:

- Tener un objetivo claro.
- Comunicación constante.
- Monitoreo y feedback continuo.

Comparar siempre lo planificado contra lo real es esencial para reaccionar a tiempo.

Factores de Éxito y Fracaso

Éxito:

- Objetivos claros.
- Buena comunicación.
- Monitoreo constante.

Fracaso:

- Problema mal definido.
- Planificación sin datos suficientes.
- Plan de proyecto pobre en detalles.
- Estimaciones basadas en suposiciones.
- Recursos mal asignados.
- Falta de un responsable claro.
- Planificación realizada sin involucrar a quienes ejecutan.
- Falta de seguimiento del plan de proyecto.